

HTML5 — Structure & sémantique

Développement web — Classe passerelle

Semaine 2 | **Module B**

Anne Terrier | cfpt·i — École d'informatique

Au programme aujourd'hui

1. Le HTML, langage de balisage
2. La structure d'un document
3. Les éléments sémantiques
4. Structurer le texte

Au milieu

Un exercice : **votre première page web.**

5. Liens & images
6. Listes & tableaux
7. Valider son HTML

Objectif de la semaine

Écrire une page HTML5 valide et sémantique, et savoir corriger les erreurs signalées par le validateur.

Où en sommes-nous ?

La semaine dernière

Le modèle client / serveur, les trois langages du navigateur, l'environnement de travail et le premier dépôt Git.

HTML

la structure

semaines 2 et 3

CSS

la présentation

semaines 4 et 5

JavaScript

le comportement

semaines 6 et 7

Aujourd'hui, on ne fait que du HTML

Votre page sera laide. C'est normal, et c'est même voulu : tant qu'il n'y a pas de style, on ne peut juger que la **structure**.

1. Le HTML, langage de balisage

Définition

Le **HTML** (*HyperText Markup Language*) décrit la **structure** et le **contenu** d'une page web. Une page est un simple fichier texte d'extension `.html`.

Ce n'est pas un langage de programmation

Ni variables, ni conditions, ni boucles : on **balise** le contenu pour dire au navigateur ce que chaque morceau représente.

Les notions de programmation arriveront avec JavaScript, en **semaine 6**.

Anatomie d'une balise

`<p> Bonjour tout le monde </p>`

↑
Balise ouvrante

↑
Contenu

↑
Balise fermante

Un élément

L'ensemble ouvrante + contenu + fermante.
La fermante se reconnaît à sa barre oblique.

Balises auto-fermantes

Sans contenu, elles s'écrivent seules :
`<img>`, `<br>`, `<meta>`, `<link>`.

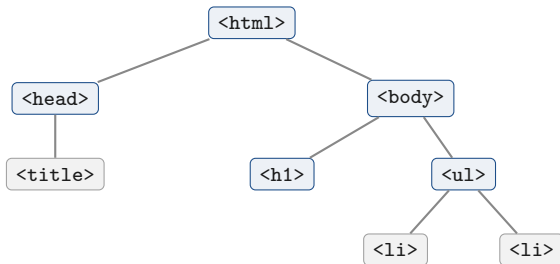
Un attribut précise le comportement d'une balise. Il s'écrit dans la balise **ouvrante**, sous la forme `nom="valeur"` :

```
<a href="https://www.hepia.ch" target="_blank">Site de la hepia</a>
```

- href indique la **destination** du lien
- target demande l'ouverture dans un **nouvel onglet**

Une balise **fermante** ne porte jamais d'attribut.

L'imbrication : un document est un arbre



Pourquoi ça compte

Cet arbre sera parcouru par le navigateur, par les moteurs de recherche, et — dès la semaine 7 — par votre code JavaScript.

La règle d'imbrication

Premier ouvert, dernier fermé

Les balises se ferment dans l'ordre inverse de leur ouverture, comme des parenthèses. Elles ne peuvent pas se chevaucher.

OUI `<p>Un <strong>mot</strong> important</p>`

NON `<p>Un <strong>mot</p></strong> important`

Le réflexe qui évite tout

Indenter : un niveau d'indentation par niveau d'imbrication. Une balise oubliée devient alors visible à l'œil nu.

Le HTML ignore votre mise en forme du texte

Espaces et retours à la ligne

Plusieurs espaces, les tabulations et les retours à la ligne sont **réduits à un seul espace** à l'affichage.

Appuyer sur Entrée ne crée pas de paragraphe : c'est un nouveau `<p>` qui le fait.

```
<!-- Un commentaire : invisible dans la page,  
      mais lisible par tous via Ctrl+U -->  
<p>Ce      texte      s'affichera      normalement.</p>
```

Un commentaire sert à **expliquer** son code, jamais à y cacher une information.

2. La structure d'un document

Le squelette d'une page HTML5

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MiniForum</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Tout le contenu visible de la page -->
</body>
</html>
```

À connaître par cœur — il sera demandé au contrôle écrit de la semaine 12.

Ce que contient le <head>

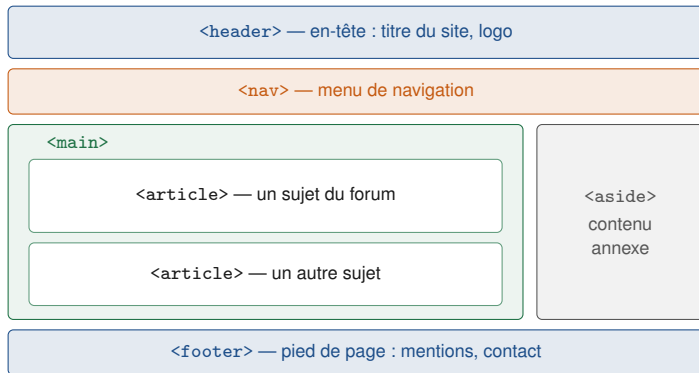
Balise	Ce qu'elle fait
<code><meta charset="UTF-8"></code>	L'encodage. Sans elle, les accents deviennent illisibles
<code><meta name="viewport"></code>	Affichage correct sur téléphone (responsive, semaine 5)
<code><title></code>	Le texte de l'onglet, des favoris et des résultats de recherche
<code><link rel="stylesheet"></code>	Rattache la feuille de style CSS

<head> et <body> : ne pas confondre

Le <head> contient des informations *sur* la page. Le <body> contient tout le **visible**. Un titre écrit dans le <head> n'apparaîtra jamais dans la page.

3. Les éléments sémantiques

Décrire le rôle de chaque zone



Pourquoi la sémantique ?

Avant HTML5, les pages n'étaient qu'un empilement de `<div>`. Les balises sémantiques disent ce que le contenu **est**, pas comment il s'affiche.

- les **humains** qui relisent le code — vous, dans trois semaines
- les **moteurs de recherche**, qui repèrent le contenu principal
- les **lecteurs d'écran**, qui permettent de sauter directement au contenu

Déroutant, mais essentiel

Remplacer un `<div>` par un `<main>` **ne change rien à l'affichage**. Le bénéfice est invisible — et bien réel. C'est aussi pourquoi le TP se fait sans CSS : rien ne doit masquer la structure.

`<section>` ou `<article>` ?

Le test

Ce contenu garderait-il du sens **sorti de la page**, publié seul ailleurs ?

Oui → `<article>`

Un message de forum, un billet de blog, une fiche produit. Autonome.

Non → `<section>`

« Sujets récents » n'a de sens que dans le forum qui la contient.

Et le `<div>` ? Il reste utile pour regrouper des éléments **uniquement** en vue de la mise en forme, quand il n'y a aucun sens à exprimer.

4. Structurer le texte

```
<h1>MiniForum</h1>
<h2>Sujets récents</h2>
<h3>Bienvenue sur le forum</h3>

<p>Un paragraphe avec du <strong>contenu important</strong>
    et un mot en <em>emphase</em>.</p>
```

 plutôt que , plutôt que <i>

 et <i> décrivent une *apparence*; et décrivent un *sens*. Le rendu est identique — mais un lecteur d'écran change d'intonation sur et ignore .

La hiérarchie des titres

Les titres construisent le **plan** du document, comme dans un rapport. Ce n'est pas un outil de mise en forme.

Plan correct

```
h1 MiniForum
  h2 Sujets récents
    h3 Bienvenue
    h3 Installer Node.js
  h2 Nouveau sujet
```

Plan incorrect

```
h1 MiniForum
  h3 Sujets récents (saut)
    h2 Bienvenue
  h1 Nouveau sujet (2e h1)
```

Deux règles : **un seul** `<h1>` par page, et **jamais de saut de niveau**. Choisir un `<h3>` « parce qu'il fait la bonne taille » est une erreur — la taille, c'est du CSS.

Éléments en bloc et éléments en ligne

En bloc

Occupe toute la largeur et provoque un retour à la ligne.

`<p>`, `<h1>`, `<ul>`, `<section>`, `<div>`

En ligne

Reste dans le fil du texte, sans le couper.

`<a>`, `<strong>`, `<em>`, `<img>`, `<span>`

Cela explique pourquoi deux paragraphes se suivent verticalement
alors que deux liens restent côte à côte.

Cette distinction deviendra centrale en semaine 4, avec le modèle de boîte du CSS.

5. Liens & images

Le lien hypertexte

```
<!-- Lien externe : URL complète -->
<a href="https://www.hepia.ch">Site de la hepia</a>

<!-- Vers un autre fichier du site -->
<a href="sujet.html">Voir le sujet</a>

<!-- Vers un élément de la page courante -->
<a href="#nouveau-sujet">Créer un sujet</a>
```

Écrire un bon intitulé

Le texte du lien doit décrire sa destination **hors contexte** : les lecteurs d'écran savent lister tous les liens d'une page, et une série de « cliquez ici » n'informe personne.

OUI Lire le sujet complet **NON** cliquez ici

Chemins relatifs : la source d'erreur n° 1

```
miniforum/  
|-- index.html  
|-- sujet.html  
|-- style.css  
\-- images/  
    |-- logo.png
```

Écriture	Signification
sujet.html	même dossier
images/logo.png	sous-dossier
../style.css	dossier parent
https://...	autre serveur

Trois pièges

La casse compte sur un serveur Linux (`Logo.png` $\neq$ `logo.png`) : ça marche chez vous, ça casse en ligne. **Pas d'espaces ni d'accents** dans les noms de fichiers. **Jamais** de chemin `C:/Users/...`

Un id donne un **nom unique** à un élément de la page :

```
<a href="#nouveau-sujet">Créer un sujet</a>  
  
<section id="nouveau-sujet"> ... </section>
```

Règle d'or

Un id est **unique** dans la page. Le validateur signale les doublons comme des erreurs.

Un point d'accroche pour l'année

Le `<label>` (sem. 3), le **CSS** (sem. 4), puis **JavaScript** (sem. 7) s'en serviront tous.

Les images

```


<figure>
  
  <figcaption>Organisation des pages du MiniForum</figcaption>
</figure>
```

L'attribut alt n'est pas optionnel

Il est lu à voix haute par les lecteurs d'écran, et s'affiche si l'image ne charge pas. Le validateur signale son absence comme une **erreur**.

OUI alt="Logo du MiniForum" **NON** alt="image"

À vous de jouer !

Exercice intermédiaire — votre première page web

Exercice — votre première page web

Objectif

Écrire seul·e une page HTML5 valide et sémantique. Pas de MiniForum ici : une courte page de présentation personnelle.

Consignes — fichier `ma-page.html`

1. Le squelette écrit **à la main** : `<!DOCTYPE>`, `<html lang>`, `<head>` avec `charset` et `<title>`, `<body>`
2. Un `<header>` avec un `<h1>` portant votre prénom
3. Un `<main>` avec une `<section>` : un `<h2>`, un paragraphe, un mot en `<strong>`
4. Une image, avec un `alt` correct
5. Un lien vers un site que vous aimez, à l'intitulé explicite
6. Un `<footer>` avec la date du jour

20 minutes — puis on valide ensemble sur validator.w3.org.

Exercice — les points d'attention

Si vous êtes en avance

- Ajouter une liste `<ul>` de trois centres d'intérêt
- Ajouter un second `<h2>` et une deuxième `<section>`
- Placer l'image dans une `<figure>` avec sa `<figcaption>`

Ce que je vérifierai en passant

- Le code est **indenté**
- **Un seul** `<h1>`
- L'image a un `alt` qui décrit quelque chose
- Le chemin de l'image est **relatif**
- Aucune balise n'est laissée ouverte

Pas de squelette généré automatiquement pour cet exercice : on l'écrit à la main, une fois.
Le raccourci ! + Tab de VS Code sera autorisé dès le TP.

Exercice — une correction possible

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Alice - présentation</title>
</head>
<body>
  <header><h1>Alice</h1></header>
  <main>
    <section>
      <h2>Qui suis-je ?</h2>
      <p>Étudiante en classe passerelle, je découvre le
        <strong>développement web</strong> cette année.</p>
      
      <p><a href="https://developer.mozilla.org">La documentation MDN</a></p>
    </section>
  </main>
  <footer><p>Page créée le 15 septembre 2026</p></footer>
</body>
</html>
```

6. Listes & tableaux

Liste à puces — ordre indifférent

```
<ul>  
  <li>HTML </li>  
  <li>CSS </li>  
  <li>JavaScript </li>  
</ul>
```

Liste numérotée — l'ordre compte

```
<ol>  
  <li>Ouvrir le terminal </li>  
  <li>Taper la commande </li>  
  <li>Valider </li>  
</ol>
```

Un `<li>` vit toujours dans un `<ul>` ou un `<ol>`

Rien d'autre qu'un `<li>` ne peut être enfant direct d'un `<ul>` : pas de `<p>`, pas de `<div>`.
Ce contenu se place à l'intérieur du `<li>`.

Un menu de navigation est une liste de liens

```
<nav>
  <ul>
    <li><a href="index.html">Accueil</a></li>
    <li><a href="sujet.html">Sujets</a></li>
  </ul>
</nav>
```

Étrange ? Pas tant que ça

Cela paraît lourd tant qu'on pense en termes d'**apparence**. Mais un menu *est* bien une liste de liens — et le CSS se chargera, en semaine 4, de l'afficher horizontalement.

C'est exactement l'esprit du HTML : **décrire le sens**, laisser le CSS décider de la forme.

```
<table>
  <caption>Derniers sujets publiés</caption>
  <thead>
    <tr><th>Sujet</th><th>Auteur</th></tr>
  </thead>
  <tbody>
    <tr><td>Bienvenue</td><td>Alice</td></tr>
    <tr><td>Installer Node.js</td><td>Bob</td></tr>
  </tbody>
</table>
```

Un tableau sert à des **données**, jamais à mettre en page

Horaire, relevé de notes, comparatif : oui. Positionner des éléments dans la page : non — c'est le rôle du CSS. Une page mise en page avec des tableaux est illisible pour un lecteur d'écran.

7. Valider son HTML

Pourquoi valider

Le navigateur est trop gentil

Il affiche une page même si des balises sont mal fermées : il **devine** ce que vous avez voulu écrire.

Et c'est un piège

Chaque navigateur devine à sa manière. Le jour où l'affichage se dérègle, l'erreur est déjà enfouie sous cent lignes de code — et elle ne se manifeste que chez certains utilisateurs.

Le validateur du W3C — `validator.w3.org`

Trois modes : par **adresse**, par **fichier** téléversé, ou par **copier-coller** — c'est celui qu'on utilise en classe.

Lire un message d'erreur

Chaque message comporte trois parties : la **ligne et la colonne**, la **description**, et un **extrait** du code fautif.

```
Error: End tag section seen, but there were open elements.  
From line 24, column 3; to line 24, column 12
```

Traduction

À la ligne 24, on ferme une `<section>` alors qu'un élément ouvert à l'intérieur ne l'a pas été. Il manque une balise fermante **au-dessus**.

Les messages les plus fréquents

Message	Ce qu'il faut corriger
End tag X seen, but there were open elements	Une balise ouverte à l'intérieur n'a pas été fermée
Unclosed element X	La balise X n'est jamais fermée
Duplicate ID "xyz"	Deux éléments portent le même id
An img element must have an alt attribute	Une image sans alt
Element X not allowed as child of element Y	Mauvaise imbrication
Stray end tag X	Une fermante sans ouvrante

Méthode — corriger de haut en bas

Une seule balise oubliée peut produire dix messages en cascade. Corrigez **la première erreur**, relancez : le compteur chute souvent d'un coup. Les **Errors** doivent toutes disparaître ; les **Warnings** sont des maladrresses.

Prévenir plutôt que corriger

Trois habitudes qui suppriment la majorité des erreurs

- **Indenter** systématiquement — Maj+Alt+F dans VS Code, avec Prettier
- Écrire la balise **fermante immédiatement** après l'ouvrante, puis remplir le contenu
- Se fier au **soulignement rouge** de VS Code, qui repère déjà une partie des erreurs

Le réflexe documentation

MDN Web Docs reste la première source : cherchez `mdn` suivi du nom de la balise. La page donne la description, la syntaxe, et surtout des **exemples modifiables**.

Ce qu'on a vu

- Le HTML, langage de balisage
- L'imbrication, et l'arbre du document
- Le squelette : `<head>` / `<body>`
- Les balises sémantiques
- Titres, texte, liens, images
- Listes et tableaux
- La validation W3C

À faire pour la semaine 3

- Les **deux pages** HTML du MiniForum (fiche TP)
- Chacune valide **sans erreur**
- Pousser le tout sur GitHub

La semaine prochaine

Les **formulaires** et l'**accessibilité** — encore du HTML, et la dernière semaine avant le CSS.

Des questions ?

Prochaine étape : la fiche TP — structure HTML du MiniForum